

03: Context Engineering

Introduction

This section explains **context engineering**, the craft of exerting deliberate control over what the model sees before it answers, as opposed to prompt engineering, which relies on carefully typing a request (and hoping for the best). This is just a reference for the concepts. Hands-on practice lives in [hands-on/](#).

What "context" means in opencode

Everything the model sees on a given turn is its context:

- your prompt,
- the files you attached with `@`,
- the contents of `AGENTS.md`,
- prior turns in the conversation,
- and (if you addressed one) a subagent's own scoped context.

Context engineering is choosing and shaping those inputs with purpose:

- `@` **mentions** — pulling exact files into context instead of relying on the model to guess or search ([02-context-modes-and-mentions.md](#)).
- `AGENTS.md` — persistent, always-on context about the project's conventions, so you don't have to restate them every prompt ([01-getting-started.md](#)).
- **Prompt structure** — how you phrase and order the instruction itself, including whether you give examples.
- **Subagents** — handing a task to an agent with its own isolated context window, so your main session isn't cluttered with exploration output it doesn't need.

The rest of this file focuses on prompt structure: **zero-shot** vs **few-shot** prompting.

Zero-shot prompting

A zero-shot prompt gives the model an instruction and relevant context, but no worked examples of the output you want. It relies entirely on the model's general knowledge and whatever's in `AGENTS.md` + attached files to infer the right shape of the answer.

```
@src/main/java/com/atlas/inventory/InventoryItem.java Write a Javadoc comment for the quote() method explaining what it escapes and why.
```

This works well when the task is common enough (Javadoc, a standard refactor, a well-known bug pattern) that the model doesn't need to see a sample to know the expected format. Most of the prompts in [02-context-modes-and-mentions.md](#) are zero-shot.

Zero-shot starts to break down when:

- you need the output to be in a specific, non-obvious format (e.g. a particular commit message style, a custom JSON error shape, a house code style),
- there are multiple reasonable ways to do the task and you want a specific one, or
- you've noticed the model's default answer keeps landing slightly wrong in the same way. That's the signal to add examples.

Few-shot prompting

A few-shot prompt includes one or more worked examples of input → output *before* asking for the real one. The examples teach the model the pattern instead of you describing it in the abstract.

For example, instead of describing the error-response shape in words, show it:

```
@src/main/java/com/atlas/inventory/InventoryServer.java
This server returns JSON error bodies in this shape when a request is invalid:
```

```
Input: quantity = -5
Output: {"error": "quantity must not be negative"}
Input: partNumber already exists ("WIDGET-100")
Output: {"error": "part number 'WIDGET-100' already exists"}
```

```
Now add a new validation: reject a reorderLevel greater than quantity * 10.
Follow the exact same error message and JSON shape as the examples above.
```

Two examples were enough here to fix the message tone ("must not be...", lower case, no trailing period) and the exact key name ("error") — properties that a zero-shot prompt would have left the model to guess at.

A few-shot prompt doesn't have to hand-write its examples every time, though. Pointing at real, existing code with `@` and saying "follow this exact pattern" is few-shot prompting too:

```
@src/main/java/com/atlas/inventory/InventoryItem.java The quote() method here is one example of this codebase's escaping style. Apply the same escaping style to the new exportToCsv() field-escaping helper you're about to write for InventoryRepository.
```

Choosing between them

A useful default: start zero-shot. If the result's substance is correct but the *shape* is off, don't argue about it in prose, but show examples of the required shape instead.

	Zero-shot	Few-shot	
Prompt length	Short	Longer	— costs more context
Best for	Common, well-understood tasks	Specific formats, house conventions, cases where output is 'off'	
Where the "examples" live	N/A	Inline in the prompt, or <code>@</code> -referenced existing code	

This is usually faster than a paragraph of formatting instructions, and it's the same principle `AGENTS.md` uses at the project level: showing the model real conventions instead of describing them abstractly.

Where to practice this

Hands-on exercises applying zero-shot and few-shot prompting to real tasks in this codebase are in [hands-on/](#).